

21. Merge Two Sorted Lists Easy

Acceptance: 67.8% | [LeetCode](#)

You are given the heads of two sorted linked lists `list1` and `list2`.

Merge the two lists into one **sorted** list. The list should be made by splicing together the nodes of the first two lists.

Return *the head of the merged linked list*.

Example 1:

```
**Input:** list1 = [1,2,4], list2 = [1,3,4]
**Output:** [1,1,2,3,4,4]
```

Example 2:

```
**Input:** list1 = [], list2 = []
**Output:** []
```

Example 3:

```
**Input:** list1 = [], list2 = [0]
**Output:** [0]
```

Constraints:

- The number of nodes in both lists is in the range `[0, 50]`.
- `-100 <= Node.val <= 100`
- Both `list1` and `list2` are sorted in **non-decreasing** order.

Claude Opus Solution

Merge Two Sorted Lists

1. Intuition

The key insight is that both input lists are **already sorted**. This is the classic **merge step** from Merge Sort.

Since both lists are sorted, the smallest unused element in the merged result must be at the **head** of one of the two lists. So we just compare the two front nodes, pick the smaller one, advance that list, and repeat.

The clever trick to simplify the code is the **dummy head** node. Without it, we'd need special-case logic to handle the first node (because we don't initially know which list it comes from). The dummy node gives us a fixed starting point to attach nodes to, so the loop body stays uniform.

2. Approach (Pseudocode)

- Create a `dummy_head` node (placeholder).
- Let `current` point to `dummy_head` (our "tail builder").
- While BOTH lists have nodes:
 - Compare `list1.val` and `list2.val`
 - Attach the smaller node to `current.next`
 - Advance the chosen list's pointer
 - Move `current` forward
- One list is now empty; attach the remaining (non-empty) list directly, since it's already sorted. (At most one is non-empty.)
- Return `dummy_head.next` (skips the placeholder).

3. Python Solution

```
from typing import Optional

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def mergeTwoLists(
        self, list1: Optional[ListNode], list2: Optional[ListNode]
    ) -> Optional[ListNode]:
        # `dummy_head` is a placeholder so we never special-case the first node.
```

```

# `current_node` is the "tail" of the merged list we are building.
# Both start pointing at the same dummy node.
dummy_head = current_node = ListNode()

# Walk both lists simultaneously while neither is exhausted.
while list1 and list2:
    # The smaller front node becomes the next node in our result.
    if list1.val <= list2.val:      # '<=' keeps it a stable merge
        current_node.next = list1  # splice list1's node in
        list1 = list1.next         # advance list1
    else:
        current_node.next = list2  # splice list2's node in
        list2 = list2.next         # advance list2

    # Move the tail pointer forward to the node we just attached.
    current_node = current_node.next

# Exactly one list may still have nodes left (or both could be empty).
# Since each is sorted, we can attach the remainder all at once.
current_node.next = list1 if list1 else list2

# dummy_head.next is the real head of the merged list.
return dummy_head.next

```

Note: I used `<=` instead of `<` to make the merge **stable** (nodes from `list1` come before equal-valued nodes from `list2`). For correctness of *sortedness* alone, either works.

4. Complexity Analysis

Time: $O(n + m)$, where n and m are the lengths of `list1` and `list2`. - Each node is visited and spliced exactly once. The `while` loop runs until one list empties, then the leftover tail is attached in $O(1)$ (just a pointer assignment, no traversal).

Space: $O(1)$ - We only use a constant number of pointers (`dummy_head`, `current_node`). We **reuse existing nodes** by re-linking them rather than allocating new ones, so no extra space scales with input size. - (Note: the *output* list itself isn't counted as extra space since we reuse the input nodes.)

5. Alternative Approaches

A) Recursive Merge

Elegant and concise, mirrors the merge-sort recurrence:

```

class Solution:
    def mergeTwoLists(self, list1, list2):
        # Base cases: if one list is empty, the answer is the other.
        if not list1:
            return list2
        if not list2:
            return list1

        # Pick the smaller head, then recursively merge the rest.
        if list1.val <= list2.val:
            list1.next = self.mergeTwoLists(list1.next, list2)
            return list1
        else:
            list2.next = self.mergeTwoLists(list1, list2.next)
            return list2

```

- **Time:** $O(n + m)$. **Space:** $O(n + m)$ due to the recursion call stack (one frame per node in the worst case). For this problem (≤ 50 nodes each) it's safe, but on huge inputs it risks a stack overflow — the iterative version is generally preferred.

B) Without a Dummy Node

You can merge purely with pointers, but you must explicitly pick the head first and handle null cases, making the code more error-prone. The dummy-node pattern is the cleaner, idiomatic choice.

C) Collect-and-Sort (anti-pattern)

Dump all values into a list, sort, rebuild. This is $O((n+m) \log(n+m))$ time and $O(n+m)$ space — strictly worse, and it ignores the fact that the inputs are already sorted. Mentioned only to highlight why we *shouldn't* do it.

Key Takeaway

This is a fundamental **two-pointer merge** pattern. The **dummy head** technique recurs throughout linked-list problems (e.g., removing nodes, partitioning) — master it once and reuse it everywhere.

Codex Solution

1. Intuition

Both linked lists are already sorted in **non-decreasing order**.

That means we can merge them the same way we merge two sorted arrays:

- Compare the current node of `list1` and the current node of `list2`
- Take the smaller node
- Move that list's pointer forward
- Repeat until one list is empty
- Attach the remaining part of the non-empty list

The key pattern here is the **two pointers technique**.

To make linked list construction easier, we use a **dummy head node**.

The dummy node helps avoid special handling for the first inserted node.

2. Approach

Pseudocode

```
create dummy node
current = dummy

while list1 is not empty and list2 is not empty:
    if list1.val <= list2.val:
        current.next = list1
        move list1 forward
    else:
        current.next = list2
        move list2 forward

    move current forward

if list1 is not empty:
    current.next = list1
else:
    current.next = list2

return dummy.next
```

3. Python Solution

```
from typing import Optional

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def mergeTwoLists(
        self,
        list1: Optional[ListNode],
        list2: Optional[ListNode]
    ) -> Optional[ListNode]:

        # Dummy node acts as a fake starting point.
```

```

# This avoids special cases for initializing the merged list's head.
dummy = ListNode()

# current always points to the last node in the merged list so far.
current = dummy

# Continue while both lists still have nodes.
while list1 and list2:

    # If list1's current value is smaller or equal,
    # attach list1's current node to the merged list.
    if list1.val <= list2.val:
        current.next = list1

        # Move list1 forward because we used its current node.
        list1 = list1.next

    # Otherwise, attach list2's current node.
    else:
        current.next = list2

        # Move list2 forward because we used its current node.
        list2 = list2.next

    # Move current forward to the node we just attached.
    current = current.next

# At this point, at least one list is empty.
# Since both lists are sorted, the remaining nodes can be attached directly.
if list1:
    current.next = list1
else:
    current.next = list2

# The real head of the merged list is after the dummy node.
return dummy.next

```

4. Complexity Analysis

Let:

- `n` be the number of nodes in `list1`
- `m` be the number of nodes in `list2`

Time Complexity

$O(n + m)$

Each node from both lists is visited at most once.

Space Complexity

$O(1)$

We do not create new list nodes, except for one dummy node.

The merge is done by rearranging existing `next` pointers.

5. Alternative Approaches

1. Recursive Merge

We can solve this recursively:

- Pick the smaller head node
- Recursively merge the rest of that list with the other list

Example idea:

```

if not list1:
    return list2
if not list2:

```

```
    return list1

if list1.val <= list2.val:
    list1.next = mergeTwoLists(list1.next, list2)
    return list1
else:
    list2.next = mergeTwoLists(list1, list2.next)
    return list2
```

However, recursion uses extra call stack space.

Recursive Complexity

```
Time Complexity:  $O(n + m)$ 
Space Complexity:  $O(n + m)$ 
```

The iterative dummy-head solution is usually preferred because it uses constant extra space.